Handoff 46: Rate-limit counters in the database (master run WE7, ENG-11)

Date: 2026-11-09 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#54), not merged, not deployed. **One additive migration, 0010.** No new dependency, no new setting.

> **How to use this file:** upload it with handoff 06 (rate limits) and `docs/plan/feedback-security.md`. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing ENG-11 for Python Academy: rate-limit counters move from one process's memory to a
database table, so several API instances would share them. Limits and messages are unchanged.
Check that counting is atomic on PostgreSQL and SQLite, that the table can't identify anyone, that
tester feedback stays anonymous, and that a database failure doesn't switch the limits off.
```

Run plan details

Field	Value
Stage	WE7, third part: ENG-11
Base	`main`, at `1dc4b59`, after #51 merged
Part 2 boxes relied on	**None**
Migration number	**0010, revising 0008.** Decision 29 keeps 0009 for the sign-ups switch (S04); question 1
Merge order	Any. Conflicts only in `docs/run-log.md`, with other PRs from tonight

1. Step 0

Open: #52 and #53 (tonight). Worktree `../python-academy-eng11`; the main folder was not switched.

2. Handoff record

Field	Value
Task	Move rate-limit counters to the database, with behaviour identical to today
Human goal	The API can run as more than one instance without each one having its own limits
Non-goals	Changing any limit, message or which address counts (`CLIENT_IP_HEADER`, `TRUSTED_PROXY_HOPS`)
Data and compatibility	New table `rate_limits`. The previous version keeps counting in memory during a deploy
Deployment status	Not requested. The migration runs at start-up, as the others do

Files changed

File	Change
`backend/migrations/versions/0010_rate_limits.py`	New table: `key` (32 hex characters), `window_start` (milliseconds), `count`
`backend/app/db.py`	`RateCount` model
`backend/app/limits.py`	Counting in the database, one atomic statement per window; memory for private rules and when the database fails; wall-clock time; the hash key from `SESSION_SECRET`
`backend/app/feedback.py`	Its docstring: where feedback's counters live
`backend/tests/test_limits.py`, `test_migrations.py`	8 new tests; 2 older ones now look in the table

3. Security and privacy note

A row can't name anyone. Its key is an HMAC of the rule, the window and `u:<id>` or `ip:<address>`, keyed by a value derived from `SESSION_SECRET`. A test checks the table holds no address, user ID, email, lesson or code.

Tester feedback stays anonymous. Someone with the database **and** `SESSION_SECRET` could test a user ID against a key. For most rules that tells them nothing the `attempts` and `events` tables don't. For feedback it would show who sent feedback, and roughly when, which `feedback-security.md` says must never be possible. **So feedback's per-tester counters stay in memory** (`private=True`), as before. Its total for everyone together, which names no one, is in the database. The cost: with several instances, a tester's own feedback limit is per instance.

A database failure doesn't switch limits off: that request is counted in memory instead, and the log says `counting in memory: rule=<name>`, nothing else.

One weakness found and fixed while building it: the hash key was read when the module loaded. If `SESSION_SECRET` were set later, it would have been made from an empty value. It is now read each time it's used.

4. Decisions and why

Decision	Options considered	Reason	Who decided
Always the database, with memory as the fallback	A setting to choose	No new setting for Lucas to manage (plan section 1.6)	Implementer
`INSERT ... ON CONFLICT DO UPDATE ... RETURNING`, one per window	Read, then write	Two instances counting at once can't both miss a request	Implementer
Feedback per tester in memory	Database too	Anonymity (section 3)	Implementer
Wall-clock time	Monotonic time, as before	Instances must agree when a window started	Implementer
Delete rows older than the longest window, every 200 requests per instance	A scheduled job	No new moving part	Implementer
Migration 0010	Take 0009	Decision 29	Plan

5. Bugs found and how they were fixed

- **The hash key could come from an empty** `SESSION_SECRET` (section 3). Found by a test where two "instances" didn't share counts.
- **My own test expected the wrong wait** after a minute window restarted while the hour window was full. Both databases gave the right answer, 3,539 seconds. The test was corrected.

6. How it was tested

Command	Result
Backend, SQLite	**902 passed, 12 skipped** before the last test was added
Backend with `TEST_POSTGRES_URL` (local PostgreSQL 16)	**916 passed**
Frontend	Not changed. CI runs it

New tests: the table holds only hashes; two instances share one limit; a restart doesn't reset counters; a window starts again; feedback per tester never reaches the table; a database failure still limits; old rows are deleted; the counting statement on **both SQLite and PostgreSQL**; migration 0010 up and down, keeping user data.

Deliberate-break controls (each undone; the file matched its saved copy byte for byte afterwards)

What I broke	Tests that failed
B1. A window never starts again	**6**
B2. Feedback per tester goes to the database	**1**
B3. No memory fallback when the database fails	**1**
B4. Each process with its own hash key again	**4**

7. Known limitations

- **A small change from today:** counters survive a restart or redeploy. That is the point of moving them.
- **Every limited request writes to the database,** one to three small statements. Fine at this size.
- **Feedback per tester is per instance** if there are several (section 3).

8. Live checks for you

After deploying: Render's log should show "Database schema is at migration 0010" once it has started. Use the app as normal. If you see "counting in memory" in the log, the database had a problem.

9. Questions for you

1. **Migration numbering.** This is 0010, revising 0008, because 0009 is kept for the sign-ups switch. When S04 is built, its 0009 will revise 0010. *Recommended: accept that, or tell me to renumber this one to 0009 and give S04 0010.*

10. For the docs sync

Where	What
Backlog	ENG-11 `in_review` (#54, handoff 46)
Stage table	"46. Rate limits in the database | ENG-11 | in_review (#54, handoff 46)"
Decision log	Feedback's per-tester counters stay in memory, for anonymity

11. Next concrete action

Answer question 1, review and merge.

12. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-eng11 status --short   # expect nothing
git worktree remove ../python-academy-eng11
git worktree prune
git branch -d eng/db-rate-limits
```